# Java 17 Features

Complete Interview Preparation Guide

This document covers all major Java 17 features with:

**✓ Simple Explanations**
**✓ Easy Code Examples**
**✓ Interview Questions & Answers**
**✓ Common Mistakes & Notes**
**✓ JDK 11 vs JDK 17 Differences**

## Quick Introduction

Java 17 was released on September 15, 2021, and is a Long-Term Support (LTS) release — the second LTS version after Java 11. It is mandatory for Spring Boot 3.0+ applications, making it extremely important for modern Java developers. Java 17 brings major language features like Sealed Classes, Pattern Matching, Text Blocks, and Records as stable (non-preview) features.

Java 17 will receive support for at least 8 years, making it the go-to version for enterprise applications moving away from Java 8 or Java 11. Understanding its features is essential for interviews, since many teams are actively migrating to Java 17.

# Table of Contents


1. 1. Sealed Classes & Interfaces
2. 2. Pattern Matching for instanceof
3. 3. Pattern Matching for switch (Preview)
4. 4. Text Blocks
5. 5. Records
6. 6. Enhanced Pseudo-Random Number Generators
7. 7. Strongly Encapsulate JDK Internals
8. 8. Foreign Function & Memory API (Incubator)
9. 9. Vector API (Second Incubator)
10. 10. Context-Specific Deserialization Filters
11. 11. Restore Always-Strict Floating-Point Semantics
12. 12. New macOS Rendering Pipeline & AArch64 Port
13. 13. Removed & Deprecated Features
14. 14. Migration Guide: JDK 11 to JDK 17

# 1. Sealed Classes & Interfaces

## Brief Explanation

Sealed Classes let you control which classes can extend or implement a particular class or interface. Before Java 17, any class could extend any other class as long as it was not marked 'final'. Sealed classes add a middle ground — only the classes you explicitly permit can extend it.

## Summary

- Use the 'sealed' keyword on a class or interface
- Use 'permits' keyword to list allowed subclasses
- Subclasses must be: final, sealed, or non-sealed
- sealed class and permitted subclasses must be in the same package (or module)
- Finalized in Java 17 (preview in Java 15 & 16)
- Works great with Pattern Matching for switch

## Detailed Description

Before sealed classes, Java had two extremes: a class open to any extension, or a 'final' class that cannot be extended at all. Sealed classes allow a class author to say exactly who can extend them. This enables the compiler to know all possible subclasses, which makes pattern matching exhaustive and safer. It is heavily used in domain modeling (e.g., shapes, payment types, AST nodes).

## Simple Examples

Basic sealed class:

```
// Sealed class — only Circle, Square, Triangle can extend Shape
public sealed class Shape
    permits Circle, Square, Triangle { }

// Must be final, sealed, or non-sealed
public final class Circle extends Shape {
    double radius;
}

public non-sealed class Square extends Shape {
    // non-sealed = any class can extend Square
    double side;
}

public sealed class Triangle extends Shape
    permits EquilateralTriangle { }

public final class EquilateralTriangle extends Triangle { }
```

Sealed interface:

```
public sealed interface Payment
    permits CreditCard, PayPal, Cash { }

public record CreditCard(String number) implements Payment { }
public record PayPal(String email) implements Payment { }
public final class Cash implements Payment { }
```

Using sealed classes with switch (compiler knows all cases):

```
static double area(Shape s) {
    return switch (s) {
        case Circle c    -> Math.PI * c.radius * c.radius;
        case Square sq   -> sq.side * sq.side;
```

```
        case Triangle t  -> 0; // simplified
        // No default needed — compiler knows all subtypes!
    };
}
```

## Interview Questions and Answers

### Q1: What are Sealed Classes in Java 17?

**Answer:** Sealed classes restrict which classes can extend or implement them. Using the 'sealed' keyword and the 'permits' clause, a class author explicitly controls the inheritance hierarchy. Subclasses must be declared as final, sealed, or non-sealed.

### Q2: What are the three options for a subclass of a sealed class?

**Answer:** A permitted subclass must be: (1) final — cannot be extended further, (2) sealed — can be extended, but only by its own permitted list, or (3) non-sealed — open to extension by any class, breaking the seal.

### Q3: Why were Sealed Classes introduced?

**Answer:** To give developers control over class hierarchies, enabling the compiler to know all possible subtypes. This enables exhaustive pattern matching in switch expressions (no default needed when all cases are covered) and improves type safety.

### Q4: Can a sealed class be abstract?

**Answer:** Yes. A sealed class can be abstract. This is very common — for example, 'public abstract sealed class Shape permits Circle, Square' forces subclasses to define shape-specific behavior.

### Q5: Where must permitted subclasses be defined?

**Answer:** In the same package as the sealed class (or in the same module if using the module system). They cannot be in a different package.

## Common Mistakes & Important Notes

⚠ Forgot 'permits'? The compiler requires it — you cannot leave the permitted list implicit.
⚠ Every permitted subclass must be in the same package (or module) as the sealed class.
⚠ A non-sealed subclass breaks the seal — any class can then extend it. Use carefully.
⚠ Sealed classes work beautifully with Records — Records are implicitly final and perfect as sealed subclasses.
⚠ JDK 11 vs 17: Sealed classes did NOT exist in Java 11. This is a purely Java 17 feature.

# 2. Pattern Matching for instanceof

## Brief Explanation

Pattern Matching for instanceof (finalized in Java 17, introduced in Java 16) removes the need for explicit casting after an instanceof check. You can declare a variable of the matched type directly in the instanceof expression itself.

## Summary

- Eliminates redundant explicit casting after instanceof check
- The pattern variable is automatically cast and scoped
- Introduced as preview in Java 14, finalized in Java 16 (available in Java 17+)
- The pattern variable is only in scope when the condition is true
- Makes code shorter, safer, and easier to read

## Simple Examples

Old way (before Java 17):

```
Object obj = "Hello Java 17";

// Old approach — check then cast (redundant)
if (obj instanceof String) {
    String s = (String) obj;  // explicit cast needed
    System.out.println(s.length());
}
```

New way (Java 17 pattern matching):

```
// New approach — check and cast in one step
if (obj instanceof String s) {
    System.out.println(s.length()); // s is available here
}
// s is OUT OF SCOPE here
```

Using with negation:

```
// Pattern variable available in else branch when negated
if (!(obj instanceof String s)) {
    System.out.println("Not a string");
} else {
    System.out.println(s.toUpperCase()); // s is in scope here
}
```

Practical example with multiple types:

```
static void printInfo(Object obj) {
    if (obj instanceof Integer i) {
        System.out.println("Integer: " + i * 2);
    } else if (obj instanceof String s) {
        System.out.println("String length: " + s.length());
    } else if (obj instanceof Double d) {
        System.out.println("Double: " + d);
    }
}
```

## Interview Questions and Answers

### Q1: What is Pattern Matching for instanceof in Java 17?

**Answer:** It allows combining the instanceof type check and variable binding in a single expression. Instead of checking with instanceof then casting, you write 'obj instanceof String s' which checks the type AND binds the object to 's' automatically if the check passes.

### Q2: What is the scope of the pattern variable?

**Answer:** The pattern variable is in scope only where the compiler can guarantee the check was true. In an 'if (obj instanceof String s)' block, 's' is in scope inside the if block. Outside it or in the else block (unless negated), it is not in scope.

### Q3: How is this different from Java 11?

**Answer:** Java 11 had no pattern matching for instanceof. You had to do a two-step process: check with instanceof, then cast manually. Java 17 combines both into a single expression, eliminating the redundant cast.

### Q4: Can you add conditions to the pattern match?

**Answer:** Yes, you can combine with &&: 'if (obj instanceof String s && s.length() > 5)'. The pattern variable 's' is in scope because of short-circuit evaluation — if the instanceof check fails, the second part won't run.

## Common Mistakes & Important Notes

- ⚠ The pattern variable is NOT the same as a regular variable — it's scoped by the compiler's flow analysis.
- ⚠ You cannot reassign the pattern variable in Java 17 (it is effectively final in many contexts).
- ⚠ This was a PREVIEW feature in Java 14 and 15 — it only became STABLE in Java 16/17.
- ⚠ Java 11 vs 17: Pattern matching for instanceof did NOT exist in Java 11.

# 3. Pattern Matching for switch (Preview in Java 17)

## Brief Explanation

Pattern Matching for switch extends the switch statement and expression to allow patterns (type patterns, null handling, guarded patterns) in case labels. Before Java 17, switch only worked with a few types (int, String, enum). Now it can match any Object type with patterns.

## Summary

- Available as a Preview feature in Java 17 (JEP 406)
- Switch can now accept any reference type (Object, interfaces, etc.)
- Type patterns in case labels: 'case String s ->'
- Null handling: 'case null ->' is now supported
- Guarded patterns with 'when' (added in Java 21): 'case Integer i when i > 0 ->'
- Works with sealed classes for exhaustive pattern matching (no default needed)

## Background: Switch Evolution

| Java Version | Switch Feature | Status |
|---|---|---|
| Java 8 and before | Traditional switch with break | Stable |
| Java 12 | Arrow case syntax (case X -> result) | Preview → Stable in Java 14 |
| Java 14 | Switch Expressions (yield keyword) | Stable |
| Java 17 | Pattern Matching for switch (type patterns, null case) | Preview |
| Java 21 | Pattern Matching for switch + guarded patterns (when clause) | Stable |

## Simple Examples

Traditional switch (old way, before Java 12):

```
// Old — verbose, uses break, only int/String/enum
static String getDayType(int day) {
    String type;
    switch (day) {
        case 1: case 7: type = "Weekend"; break;
        default:        type = "Weekday"; break;
    }
    return type;
}
```

Arrow syntax (since Java 12/14):

```
// Cleaner — no break, can be an expression
static String getDayType(int day) {
    return switch (day) {
        case 1, 7 -> "Weekend";
        default   -> "Weekday";
    };
}
```

Pattern Matching for switch (Java 17 Preview):

```
// Now works with any Object type
static String format(Object obj) {
    return switch (obj) {
        case null           -> "null value";
        case Integer i      -> "Integer: " + i;
        case Long l         -> "Long: " + l;
        case Double d       -> "Double: " + d;
        case String s       -> "String: '" + s + "'";
        default             -> "Unknown: " + obj.toString();
    };
}
```

With sealed classes — no default needed:

```
sealed interface Shape permits Circle, Square { }
record Circle(double r) implements Shape { }
record Square(double side) implements Shape { }

static double area(Shape s) {
    return switch (s) {
        case Circle c  -> Math.PI * c.r() * c.r();
        case Square sq -> sq.side() * sq.side();
        // No default needed! Compiler knows all cases.
    };
}
```

Multi-value case labels:

```
int value = 5;
switch (value) {
    case 1      -> System.out.println("One");
    case 2      -> System.out.println("Two");
    case 3, 4   -> System.out.println("Three or Four");
    default     -> System.out.println("Other");
}
```

## Interview Questions and Answers

### Q1: What is Pattern Matching for switch in Java 17?

**Answer:** It is a preview feature (JEP 406) that extends switch to work with patterns in case labels. You can now match on any reference type (not just int/String/enum), handle null explicitly with 'case null', and use type patterns like 'case String s'.

### Q2: Is Pattern Matching for switch stable in Java 17?

**Answer:** No. It was a PREVIEW feature in Java 17. It became stable in Java 21. You need --enable-preview flag to use it in Java 17. This is a very common interview trick question.

### Q3: What is the difference between 'case X:' and 'case X ->' in switch?

**Answer:** The old 'case X:' syntax has fall-through behavior — execution continues to the next case unless you add 'break'. The new 'case X ->' syntax (since Java 12) has NO fall-through. It is cleaner and safer. The arrow syntax also allows using switch as an expression.

### Q4: What is the 'yield' keyword in switch?

**Answer:** yield is used inside a switch expression (not switch statement) to return a value from a block. Example: case 1 -> { int x = compute(); yield x; }. It replaces the need for a temporary variable outside the switch.

### Q5: How does switch Pattern Matching work with sealed classes?

**Answer:** When switching on a sealed type, the compiler knows all permitted subtypes. If you cover all of them in case labels, no default is required. The compiler verifies exhaustiveness at compile time — a huge safety improvement.

## Common Mistakes & Important Notes

⚠ Pattern Matching for switch is PREVIEW in Java 17 — needs --enable-preview to compile/run.
⚠ Guarded patterns with 'when' clause were added in Java 21, NOT Java 17.
⚠ 'case null' handling is a new Java 17 preview feature — without it, switch throws NullPointerException on null input.
⚠ Java 11 vs 17: Arrow syntax arrived in Java 12. Pattern matching in switch arrived (as preview) in Java 17.

# 4. Text Blocks

## Brief Explanation

Text Blocks provide a way to write multi-line string literals in Java without escape sequences and string concatenation. They use triple-double-quotes (""") as delimiters and make embedded HTML, JSON, SQL, and XML strings much easier to read and write.

## Summary

- Introduced as preview in Java 13, finalized in Java 15 (available and stable in Java 17)
- Delimited by triple double-quotes: """
- Preserves formatting and indentation automatically
- Eliminates \n, \t and string concatenation for multi-line strings
- Opening """ must be followed by a newline
- Trailing whitespace is stripped by default

## Simple Examples

HTML string — old vs new:

```
// Old way (Java 8/11) — messy escape sequences
String html = "<html>\n" +
              "  <body>\n" +
              "    <p>Hello</p>\n" +
              "  </body>\n" +
              "</html>";

// New way (Java 17 Text Block) — clean and readable
String html = """
              <html>
                <body>
                  <p>Hello</p>
                </body>
              </html>
              """;
```

JSON string:

```
String json = """
        {
            "name": "Alice",
            "age": 30,
            "city": "Mumbai"
        }
        """;
```

SQL query:

```
String query = """
        SELECT id, name, salary
        FROM employees
        WHERE department = 'IT'
        ORDER BY salary DESC
        """;
```

String interpolation using formatted() — Java 17:

```
String name = "Alice";
int age = 30;
String message = """
        Hello, %s!
        You are %d years old.
```

```
        """.formatted(name, age);
System.out.println(message);
```

## Interview Questions and Answers

### Q1: What are Text Blocks in Java 17?

**Answer:** Text Blocks are multi-line string literals enclosed in triple double-quotes (""""). They were finalized in Java 15 and are fully available in Java 17. They eliminate the need for escape sequences and concatenation when embedding multi-line text like HTML, JSON, or SQL.

### Q2: What is the rule for the opening triple-quote?

**Answer:** The opening triple-quote """" must be followed immediately by a newline. Content starts on the next line. If you write """"hello, it is a compile error.

### Q3: How is indentation handled in Text Blocks?

**Answer:** Java automatically strips common leading whitespace from all lines (called incidental whitespace). The position of the closing """" determines the minimum indentation level. Lines are re-indented relative to the closing delimiter.

### Q4: How do you insert a variable into a Text Block?

**Answer:** Text Blocks are just String objects — you can use .formatted() method (added in Java 15) or String.format(). Example: """"Hello, %s"""".formatted(name).

### Q5: What is the difference between Text Blocks in Java 11 and Java 17?

**Answer:** Text Blocks did NOT exist in Java 11. They were introduced as a preview feature in Java 13 and became stable in Java 15. Java 17 includes them as a fully stable feature.

## Common Mistakes & Important Notes

- ⚠ Text Blocks are NOT available in Java 11 — they became stable only in Java 15.
- ⚠ The opening """" must be followed by a newline — inline content on the same line is a compile error.
- ⚠ Trailing whitespace on lines inside a Text Block is stripped by default.
- ⚠ Text Blocks are still String — you can call all String methods on them (strip, formatted, etc.).
- ⚠ Use \"  inside a Text Block if you need a literal double-quote character.

# 5. Records

## Brief Explanation

Records are a special kind of class in Java designed for holding immutable data. They automatically generate constructor, getters, equals(), hashCode(), and toString() methods — removing all the boilerplate code you usually write for simple data-carrier classes. Records were introduced as preview in Java 14 and finalized in Java 16 (available in Java 17).

## Summary

- Declared with 'record' keyword instead of 'class'
- All fields are implicitly final and private (immutable by default)
- Auto-generates: constructor, accessors (getters), equals(), hashCode(), toString()
- Records are implicitly final — cannot be extended
- Records can implement interfaces but cannot extend classes
- Finalized in Java 16, fully available and stable in Java 17

## Detailed Description

Before Records, creating a simple data class (like a Point with x and y coordinates) required writing: private final fields, a constructor, getters, equals(), hashCode(), and toString(). A Records replaces all of that with a single line. Records are perfect for DTOs, value objects, and sealed class hierarchies.

## Simple Examples

Old way (before Records):

```
public final class Point {
    private final int x;
    private final int y;
    public Point(int x, int y) { this.x = x; this.y = y; }
    public int x() { return x; }
    public int y() { return y; }
    @Override public boolean equals(Object o) { ... }
    @Override public int hashCode() { ... }
    @Override public String toString() { ... }
}
```

New way with Records (Java 17):

```
// One line replaces all the boilerplate above!
public record Point(int x, int y) { }

// Usage
Point p = new Point(3, 4);
System.out.println(p.x());          // 3  (accessor method)
System.out.println(p.y());          // 4
System.out.println(p);              // Point[x=3, y=4]
System.out.println(p.equals(new Point(3, 4))); // true
```

Records can have custom methods and compact constructors:

```
public record Person(String name, int age) {

    // Compact constructor — for validation
    Person {
        if (age < 0) throw new IllegalArgumentException("Age cannot be negative");
        name = name.trim(); // can transform fields
    }

    // Custom instance method
```

```
    public String greeting() {
        return "Hello, I am " + name;
    }
}
```

Records implementing interfaces (works with sealed classes):

```
sealed interface Shape permits Circle, Rectangle { }
record Circle(double radius)    implements Shape { }
record Rectangle(double w, double h) implements Shape { }

Shape s = new Circle(5.0);
if (s instanceof Circle c) {
    System.out.println("Area: " + Math.PI * c.radius() * c.radius());
}
```

## Interview Questions and Answers

### Q1: What are Records in Java 17?

**Answer:** Records are a special class type for immutable data carriers. Declared with the 'record' keyword, they automatically generate a canonical constructor, accessor methods (not traditional getters — they match field names), equals(), hashCode(), and toString(). They were finalized in Java 16 and are fully stable in Java 17.

### Q2: Can a Record extend another class?

**Answer:** No. Records implicitly extend java.lang.Record and since Java does not support multiple class inheritance, a Record cannot extend any other class. However, Records CAN implement interfaces.

### Q3: Are Record fields mutable?

**Answer:** No. All Record component fields are implicitly private and final. You cannot change them after construction. Records are designed to be immutable value objects.

### Q4: What is a compact constructor in Records?

**Answer:** A compact constructor is a special form of constructor in Records that does not list parameters — it implicitly receives all components. It is used for validation or normalization. Example: 'Person { if (age < 0) throw ...; }' — no parameter list, but all components are available.

### Q5: How are accessor methods different from getters in Records?

**Answer:** In a regular class, getters are typically named getX(). In Records, accessor methods have the same name as the field — so it's x() not getX(). This is an important distinction and a common interview question.

## Common Mistakes & Important Notes

⚠ Records are NOT available in Java 11 — they became stable in Java 16 (preview in Java 14/15).
⚠ Accessor methods in Records are fieldName() — NOT getFieldName(). Example: point.x() not point.getX().
⚠ Records are implicitly final — you cannot extend a Record.
⚠ Records cannot have non-final instance fields — all fields defined in the header are final.
⚠ You can add static fields and methods to Records.

# 6. Enhanced Pseudo-Random Number Generators (JEP 356)

## Brief Explanation

Java 17 introduces a new RandomGenerator interface and several new PRNG (Pseudo-Random Number Generator) algorithm implementations. This makes it easier to switch between different random number algorithms, use them in streams, and support thread-safe random number generation.

## Summary

- New interface: java.util.random.RandomGenerator
- New algorithms: Xoroshiro128PlusPlus, Xoshiro256PlusPlus, L32X64MixRandom, etc.
- RandomGeneratorFactory allows discovering and instantiating PRNG algorithms by name
- Better stream support: ints(), longs(), doubles() return streams
- SplittableRandom improved — supports splitting for parallel/concurrent usage
- Old classes (Random, ThreadLocalRandom) now implement the new interface

## Simple Examples

```
import java.util.random.RandomGenerator;
import java.util.random.RandomGeneratorFactory;

// Use a specific algorithm by name
RandomGenerator rng = RandomGeneratorFactory.of("Xoroshiro128PlusPlus").create();
System.out.println(rng.nextInt(100));   // random int 0-99
System.out.println(rng.nextDouble());   // random double 0.0 - 1.0

// List all available algorithms
RandomGeneratorFactory.all()
    .map(RandomGeneratorFactory::name)
    .forEach(System.out::println);

// Stream support
rng.ints(5, 1, 101)   // 5 random ints between 1 and 100
   .forEach(System.out::println);

// SplittableRandom (for multi-threaded use)
SplittableRandom sr = new SplittableRandom();
int num = sr.nextInt(10, 20); // random int between 10-19
```

## Interview Questions and Answers

### Q1: What is the main improvement to random number generation in Java 17?

**Answer:** Java 17 introduces the RandomGenerator interface in java.util.random, which all PRNG implementations (including old classes like Random) now implement. This allows writing code that is algorithm-agnostic. RandomGeneratorFactory allows discovering and selecting algorithms by name.

### Q2: What is the difference between Random and SplittableRandom?

**Answer:** Random is thread-safe but slower due to atomic operations. SplittableRandom is faster and designed for parallel use — it can be 'split' into independent sub-generators for each thread, avoiding contention. SplittableRandom should be preferred for multi-threaded applications.

### Q3: How do you list all available PRNG algorithms in Java 17?

**Answer:** Use RandomGeneratorFactory.all() which returns a Stream of all available factories. You can then map to their names with .map(RandomGeneratorFactory::name).

## Common Mistakes & Important Notes

- ⚠ The new RandomGenerator interface is in java.util.random — a new package in Java 17.
- ⚠ Old java.util.Random still works but now implements RandomGenerator.
- ⚠ SplittableRandom is NOT thread-safe on a single instance — split it for each thread.
- ⚠ Java 11 vs 17: This new PRNG API did not exist in Java 11.

# 7. Strongly Encapsulate JDK Internals (JEP 403)

## Brief Explanation

Java 17 strongly encapsulates all internal JDK APIs (like sun.misc.*, sun.reflect.*, etc.) and removes the --illegal-access flag that was available as a temporary workaround in Java 9-16. Any code that relies on internal JDK APIs will now fail with errors rather than warnings.

## Summary

- Removes the --illegal-access flag (was deprecated in Java 16, removed in Java 17)
- All non-critical internal APIs are now fully inaccessible
- Only sun.misc.Unsafe remains accessible as a critical exception
- Fixes security and stability risks from internal API usage
- Third-party libraries using internal APIs must update to public APIs
- Use --add-opens as a targeted, explicit workaround (per module, not global)

## What Changed from Java 11 to Java 17

| Version | --illegal-access Flag | Behavior |
|---|---|---|
| Java 9-15 | Available (default: permit) | Shows warning but allows access |
| Java 16 | Deprecated, default: deny | Throws exception by default |
| Java 17 | REMOVED | Always throws exception. --add-opens is the only workaround |

## Simple Examples

```
// This code compiled in Java 8 and ran with warnings in Java 11-16
import sun.misc.Unsafe;
Unsafe unsafe = Unsafe.getUnsafe(); // ERROR in Java 17

// Java 17 fix — use --add-opens for targeted access (temporary workaround)
// Command line:
// java --add-opens java.base/sun.misc=ALL-UNNAMED MyApp

// Better fix — use the public API alternative
import java.nio.ByteBuffer;
ByteBuffer buffer = ByteBuffer.allocateDirect(1024); // public API
```

## Interview Questions and Answers

### Q1: What is 'Strong Encapsulation of JDK Internals' in Java 17?

**Answer:** Java 17 (JEP 403) removes the --illegal-access flag and fully blocks access to internal JDK APIs. In Java 9-16, you could use --illegal-access=permit to get a warning instead of an error. Java 17 removes that flag — internal APIs now always throw errors.

### Q2: What is --add-opens and when should you use it?

**Answer:** --add-opens is a JVM flag that opens a specific module/package for reflective access. Example: --add-opens java.base/java.lang=ALL-UNNAMED. It should only be used as a TEMPORARY fix while waiting for libraries to update their code. It is not a permanent solution.

### Q3: Which internal API is still accessible in Java 17?

**Answer:** sun.misc.Unsafe is still accessible as a critical internal API because many frameworks and libraries depend on it. Oracle considers it too dangerous to remove immediately. All other non-critical internal APIs are inaccessible.

## Common Mistakes & Important Notes

⚠ --illegal-access=permit worked in Java 11 but is COMPLETELY REMOVED in Java 17. Remove it from JVM args.
⚠ Use --add-opens only as a temporary migration fix — not a permanent solution.
⚠ Update third-party libraries (Spring, Hibernate, etc.) to Java 17-compatible versions.
⚠ This is the #1 migration blocker when moving from Java 11 to Java 17.

# 8. Foreign Function & Memory API (JEP 412 — Incubator)

## Brief Explanation

The Foreign Function & Memory API (Project Panama) allows Java programs to interact with code and memory outside the JVM — calling native C libraries and managing off-heap memory — without using the old, error-prone JNI (Java Native Interface). It is in Incubator status in Java 17.

## Summary

- Incubator feature in Java 17 (JEP 412)
- Replacement for the old JNI (Java Native Interface)
- Allows calling native C/C++ functions directly from Java
- Manages off-heap memory safely with MemorySegment and MemorySession
- Much safer and easier than JNI — no C header files required
- Became stable in Java 22 (JEP 454)

## Why It Matters

JNI required writing C header files, compiling native code, and was very error-prone. The new Foreign Function & Memory API (developed under Project Panama) allows Java code to call C library functions directly using descriptors in Java code. Real-world use cases include machine learning libraries (TensorFlow), image processing (OpenCV), and OS-level APIs.

## Interview Questions and Answers

### Q1: What is the Foreign Function & Memory API in Java 17?

**Answer:** It is an Incubator API (JEP 412) that allows Java to call native code (C functions) and manage off-heap memory without using JNI. It provides safer and simpler APIs via MethodHandles and MemorySegment for native interop.

### Q2: What is the difference between JNI and the new Foreign Function API?

**Answer:** JNI requires writing C header files, compiling native code, and is very complex and error-prone. The Foreign Function & Memory API lets you call native functions using pure Java code with descriptors and handles — no C code required. It is much safer (memory-safe) and easier to use.

### Q3: What is the status of this API in Java 17?

**Answer:** It is in Incubator status in Java 17 (in the jdk.incubator.foreign module). It requires --add-modules jdk.incubator.foreign to use. It graduated to Preview in Java 19-21 and became stable in Java 22.

## Common Mistakes & Important Notes

⚠ This is Incubator in Java 17 — requires --add-modules jdk.incubator.foreign.
⚠ The API changed significantly between Java 17, 19, and 22 — do not rely on Java 17 APIs for production.
⚠ Interview focus: know what it does and why it replaces JNI, not the full API details.

# 9. Vector API (JEP 414 — Second Incubator)

## Brief Explanation

The Vector API allows Java code to express SIMD (Single Instruction, Multiple Data) computations — operations that run on multiple data elements simultaneously using special CPU hardware instructions. It is in its second incubator phase in Java 17.

## Summary

- Second Incubator in Java 17 (JEP 414)
- Enables SIMD (vector) CPU instructions from Java code
- Target use cases: scientific computing, image/signal processing, ML operations
- Much faster than scalar code for bulk numeric operations
- Still being refined — API may change in future versions
- The compiler maps vector operations to optimal CPU hardware instructions

## Interview Questions and Answers

### Q1: What is the Vector API in Java 17?

**Answer:** The Vector API (JEP 414, second incubator) allows Java developers to write code that uses SIMD (Single Instruction, Multiple Data) CPU instructions. This lets one CPU instruction process multiple data elements simultaneously, providing significant performance gains for numerical computations.

### Q2: What are typical use cases for the Vector API?

**Answer:** Scientific computing, image processing, signal processing, machine learning matrix operations, text processing, and any computation involving arithmetic on large arrays of numbers. These all benefit from processing multiple values simultaneously.

### Q3: Is the Vector API production-ready in Java 17?

**Answer:** No. It is in Incubator (second iteration) in Java 17. The API is still being finalized and may change. Use it in Java 17 only for evaluation and testing. It needs --add-modules jdk.incubator.vector to use.

## Common Mistakes & Important Notes

⚠ Vector API is Incubator in Java 17 — not for production use yet.
⚠ 'Vector' here means SIMD CPU vectors, NOT java.util.Vector (the old collection class).
⚠ Interview focus: understand what SIMD means and why it is useful, not detailed API usage.

# 10. Context-Specific Deserialization Filters (JEP 415)

## Brief Explanation

Deserialization of Java objects from untrusted sources is a well-known security vulnerability. Java 9 added the ability to set a global JVM-level deserialization filter. Java 17 (JEP 415) extends this to allow context-specific and dynamically selected filters — different filters for different parts of an application.

## Summary

- Extends JEP 290 (Java 9) which added basic deserialization filters
- Allows defining a filter factory to select filters dynamically per stream
- Different filters can be applied for different code paths or contexts
- Improves security by limiting what classes can be deserialized
- JVM-level protection against deserialization attacks

## Interview Questions and Answers

### Q1: What is deserialization filtering and why is it important?

**Answer:** Deserialization attacks occur when malicious serialized data is used to instantiate dangerous classes during Java object deserialization. Deserialization filters allow you to restrict which classes can be deserialized, rejecting dangerous classes before they are instantiated.

### Q2: What did Java 17 add to deserialization filters (over Java 9)?

**Answer:** Java 9 added a global JVM-level filter via ObjectInputFilter.Config.setSerialFilter(). Java 17 (JEP 415) adds a filter factory mechanism — a JVM-wide filter factory that can select different filters per deserialization stream context, allowing fine-grained control.

## Common Mistakes & Important Notes

⚠ This is a security feature — important to understand conceptually for interviews, not API-level details.
⚠ Deserialization of untrusted data is a CRITICAL security risk — always validate and filter.
⚠ Java 11 vs 17: Java 11 had basic deserialization filters (from Java 9). Java 17 adds dynamic context-specific filters.

# 11. Restore Always-Strict Floating-Point Semantics (JEP 306)

## Brief Explanation

Java 17 makes floating-point operations strictly follow IEEE 754 semantics by default across all platforms. Before Java 17, some floating-point operations were slightly relaxed for performance on x87 processors. This change restores predictability and consistency.

## Summary

- Makes floating-point results consistent across all hardware platforms
- Before Java 17: non-strictfp code might produce slightly different results on x87 hardware
- After Java 17: all floating-point ops behave as if strictfp is always enabled
- The 'strictfp' keyword is now effectively redundant (but still valid)
- Important for scientific, financial, and precision-critical applications

## Interview Questions and Answers

### Q1: What does 'Restore Always-Strict Floating-Point Semantics' mean?

**Answer:** It means Java 17 enforces IEEE 754 strict floating-point arithmetic everywhere, on all platforms. Before Java 17, some hardware (x87) could produce slightly different floating-point results unless you used the 'strictfp' keyword. Java 17 makes strict mode the universal default.

### Q2: What is the 'strictfp' keyword and what happens to it in Java 17?

**Answer:** strictfp was a keyword used to enforce strict IEEE 754 floating-point behavior in a class or method. In Java 17, since strict semantics are now always applied, 'strictfp' is redundant. The keyword is still valid (not removed) but has no practical effect.

## Common Mistakes & Important Notes

⚠ The 'strictfp' keyword is NOT removed in Java 17 — it compiles but has no effect.
⚠ This mainly matters for applications doing heavy floating-point math across platforms.
⚠ For most everyday applications, this change is transparent.

# 12. New macOS Rendering Pipeline & AArch64 Port

## Brief Explanation

Java 17 introduces two macOS-related improvements: (1) a new rendering pipeline for Java 2D/Swing graphics that uses Apple Metal instead of the deprecated OpenGL, and (2) full support for macOS on Apple Silicon (AArch64/ARM) processors.

## Summary — New macOS Rendering Pipeline (JEP 382)

- Old pipeline: used Apple OpenGL API (deprecated by Apple in macOS 10.14 Mojave)
- New pipeline: uses Apple Metal API for hardware-accelerated 2D graphics
- Better performance for Swing/Java 2D applications on macOS
- No change to the public API — existing Swing code works without modification

## Summary — macOS/AArch64 Port (JEP 391)

- Adds JDK support for Apple Silicon (M1, M2, M3 chips — AArch64/ARM64 architecture)
- Apple announced transition from Intel (x64) to its own ARM chips
- Java 17 is the first LTS to fully support native Apple Silicon
- Native ARM support means better performance on M-series Macs (no Rosetta emulation)

## Interview Questions and Answers

### Q1: Why did Java 17 change the macOS rendering pipeline?

**Answer:** Apple deprecated OpenGL in macOS 10.14 (Mojave) and eventually removed it. Java's Swing/2D graphics relied on OpenGL for hardware acceleration. Java 17 replaced it with Apple Metal API to ensure continued hardware-accelerated rendering on macOS without breaking existing Swing applications.

### Q2: What is the macOS AArch64 Port in Java 17?

**Answer:** Apple Silicon (M1, M2, M3 chips) uses AArch64 (ARM64) architecture, not Intel x64. Java 17 added a native JDK build for macOS/AArch64, so Java applications run natively on Apple Silicon without Rosetta 2 emulation, resulting in better performance and battery efficiency.

## Common Mistakes & Important Notes

⚠ The macOS Metal pipeline change is transparent — no code changes needed for Swing applications.
⚠ Before Java 17, Apple Silicon Macs had to run Java via Rosetta 2 emulation (slower).
⚠ Java 11 vs 17: Java 11 did not support native Apple Silicon — Java 17 does.

# 13. Removed & Deprecated Features in Java 17

## Brief Explanation

Java 17 removes several long-deprecated APIs and experimental features. Understanding what was removed is important for both migration and interviews.

| JEP | Feature | Status | Replacement / Note |
|---|---|---|---|
| JEP 398 | Applet API | Deprecated for Removal | No direct replacement; use HTML5/JavaScript |
| JEP 407 | RMI Activation | Removed | Use plain RMI or modern alternatives |
| JEP 410 | Experimental AOT & JIT Compiler | Removed | Use GraalVM for AOT/JIT |
| JEP 411 | Security Manager | Deprecated for Removal | Use OS-level security, Java modules |
| JEP 403 | --illegal-access flag | Removed | Use --add-opens for targeted access |

## Applet API — Deprecated for Removal (JEP 398)

The Applet API was deprecated since Java 9 and browsers stopped supporting Java plugins years ago. Java 17 marks it as deprecated-for-removal, meaning it will be fully removed in a future release. Developers should migrate to HTML5/JavaScript or other web technologies.

## RMI Activation — Removed (JEP 407)

RMI (Remote Method Invocation) itself is still available, but the RMI Activation mechanism (which allowed deferred activation of remote objects) was removed. It was already obsolete for many years. Regular RMI without activation still works.

## Experimental AOT and JIT Compiler — Removed (JEP 410)

Java 9 and 10 introduced experimental Ahead-of-Time (AOT) compilation (jaotc tool) and GraalVM-based JIT compiler. These were rarely used and expensive to maintain in the JDK. They were removed in Java 17. Developers wanting AOT or Graal JIT should use GraalVM directly.

## Security Manager — Deprecated (JEP 411)

The Security Manager (and its related SecurityException) has been deprecated for removal. It dates from Java 1.0 and was designed for untrusted applet environments that no longer exist. Modern security relies on OS-level sandboxing, Java module encapsulation, and container security.

## Interview Questions and Answers

### Q1: What is removed in Java 17 that was still present in Java 11?

**Answer:** Java 17 removes: RMI Activation (JEP 407), Experimental AOT/JIT compiler (JEP 410), and the --illegal-access flag (JEP 403). It also deprecates-for-removal: the Applet API (JEP 398) and Security Manager (JEP 411).

### Q2: Is RMI completely removed in Java 17?

**Answer:** No. Only the RMI Activation mechanism is removed. Standard RMI (Remote Method Invocation) is still available in Java 17. The activation mechanism was a separate, unused feature on top of RMI.

### Q3: What replaced the experimental AOT compiler removed in Java 17?

**Answer:** There is no built-in replacement in the JDK. Developers wanting AOT compilation should use GraalVM Native Image, which provides production-quality AOT compilation outside of the standard JDK.

### Q4: Why is the Security Manager being deprecated?

**Answer:** The Security Manager was designed for running untrusted Java applets in browsers — a use case that no longer exists. Modern Java applications use OS-level isolation, containerization (Docker), and the Java module system for security. Maintaining the Security Manager adds complexity without modern benefit.

## Common Mistakes & Important Notes

- ⚠ --illegal-access is GONE in Java 17. Replace with --add-opens where needed.
- ⚠ Security Manager is deprecated but NOT yet removed in Java 17 — removal comes in a later release.
- ⚠ Applet API is deprecated-for-removal in Java 17 but not yet fully removed.
- ⚠ jaotc (AOT compiler tool) is gone in Java 17 — use GraalVM Native Image instead.

# 14. Migration Guide: JDK 11 to JDK 17

## Brief Explanation

Migrating from Java 11 to Java 17 is generally smoother than the Java 8 to Java 11 migration because the removed modules crisis already happened. The main issues are: removal of --illegal-access flag, library updates for strong encapsulation, and deprecated APIs. Spring Boot 3.0+ REQUIRES Java 17.

| Issue | Java 11 Behavior | Java 17 Behavior | Fix |
|---|---|---|---|
| --illegal-access flag | Available (default: deny) | REMOVED — ignored with error | Use --add-opens or update library |
| Internal API access (sun.*) | Warning or exception | Always exception | Use public API or --add-opens |
| RMI Activation | Available | Removed | Remove usage or use plain RMI |
| Applet API | Available (deprecated) | Deprecated for removal (warning) | Migrate to HTML5/JavaScript |
| Security Manager | Available | Deprecated (warning) | Plan for future removal |
| Experimental jaotc AOT | Available | Removed | Use GraalVM Native Image |
| Spring Boot | Works with Spring Boot 2.x | Required for Spring Boot 3.x | Upgrade Spring Boot to 3.x |
| Jakarta EE | javax.* namespace | jakarta.* namespace (if using Spring Boot 3) | Update all javax imports to jakarta |

## Key New Features Available After Migrating to Java 17

- Sealed Classes (stable) — control class hierarchies
- Pattern Matching for instanceof (stable) — eliminate casts
- Records (stable) — eliminate data class boilerplate
- Text Blocks (stable) — clean multi-line strings
- Switch Expressions with arrow syntax (stable since Java 14)
- Pattern Matching for switch (preview — needs --enable-preview)

## Step-by-Step Migration Checklist

- Step 1: Update JDK to 17 and compile — fix any compile errors
- Step 2: Remove --illegal-access from JVM startup scripts
- Step 3: Fix any 'InaccessibleObjectException' with targeted --add-opens (temporary)
- Step 4: Update third-party libraries to Java 17-compatible versions (critical: Spring Boot 3, Hibernate 6)
- Step 5: If using Spring Boot 3, update javax.* imports to jakarta.* (jakarta namespace change)
- Step 6: Replace Applet API usage if any
- Step 7: Update build tools (Maven/Gradle) to versions that support Java 17
- Step 8: Run full test suite and check for behavioral changes

## Interview Questions and Answers

### Q1: What is the biggest difference when migrating from Java 11 to Java 17?

**Answer:** The biggest change is the removal of the --illegal-access flag. In Java 11, this flag (though deprecated) could suppress illegal reflective access warnings. In Java 17, it is completely removed — any code or library using internal JDK APIs will throw InaccessibleObjectException. Libraries must be updated.

### Q2: Why is Java 17 required for Spring Boot 3?

**Answer:** Spring Boot 3 (released November 2022) requires Java 17 as the minimum version and also requires Jakarta EE 9+, which uses the jakarta.* namespace instead of javax.*. Java 17 is the only LTS version available between Java 11 and Java 21, making it the natural baseline.

### Q3: What is the javax to jakarta namespace change?

**Answer:** When the Java EE specifications moved to the Eclipse Foundation, they rebranded from javax.* to jakarta.*. So Spring Boot 3 / Jakarta EE 9+ use jakarta.servlet, jakarta.persistence, jakarta.validation, etc. instead of javax.servlet, javax.persistence, javax.validation. This is a source-level breaking change.

### Q4: Is Java 17 backward compatible with Java 11 code?

**Answer:** Mostly yes. Pure Java 11 application code (without internal API usage or removed features) compiles and runs on Java 17. The main compatibility breaks are: removed --illegal-access flag, removed RMI Activation, removed AOT compiler, and strong encapsulation of JDK internals.

### Q5: What are the LTS Java versions and why does it matter?

**Answer:** LTS (Long-Term Support) versions receive security and bug fixes for extended periods (8+ years for Oracle). The LTS versions are Java 8, Java 11, Java 17, and Java 21. Most enterprises run LTS versions in production. Non-LTS versions (Java 9, 10, 12-16, 18-20) receive only 6 months of support.

## Java Version Comparison Summary

| Feature | Java 8 | Java 11 (LTS) | Java 17 (LTS) |
|---|---|---|---|
| Lambda & Streams | ✓ Introduced | ✓ Available | ✓ Available |
| HTTP Client API | ✗ Not available | ✓ Stable | ✓ Stable |
| Text Blocks | ✗ Not available | ✗ Not available | ✓ Stable (since Java 15) |
| Records | ✗ Not available | ✗ Not available | ✓ Stable (since Java 16) |
| Sealed Classes | ✗ Not available | ✗ Not available | ✓ Stable |
| Pattern Matching instanceof | ✗ Not available | ✗ Not available | ✓ Stable |
| Pattern Matching switch | ✗ Not available | ✗ Not available | Preview only |
| Switch Expressions (->) | ✗ Not available | ✗ Not available | ✓ Stable (since Java 14) |
| ZGC | ✗ Not available | Experimental | ✓ Production-ready |

| Feature | Java 8 | Java 11 (LTS) | Java 17 (LTS) |
|---|---|---|---|
| var keyword | ✗ Not available | ✓ (local vars, lambda) | ✓ Available |
| JAXB / Java EE modules | ✓ Built-in | ✗ Removed | ✗ Removed |
| --illegal-access flag | N/A | Available | ✗ Removed |
| Nashorn JS Engine | ✓ Available | Deprecated | ✗ Removed |
| Apple Silicon (AArch64) | ✗ Not supported | ✗ Not supported | ✓ Native support |
| Spring Boot 3 compatible | ✗ Not supported | ✗ Not supported | ✓ Required baseline |

## Common Mistakes & Important Notes

⚠ Spring Boot 3 REQUIRES Java 17 minimum — you cannot use Spring Boot 3 with Java 11.
⚠ jakarta.* vs javax.*: if upgrading to Spring Boot 3, ALL javax imports must become jakarta imports.
⚠ --illegal-access is fully removed in Java 17. Do not copy JVM args from Java 11 setups.
⚠ ZGC is production-ready in Java 17 (was experimental in Java 11) — good upgrade reason.
⚠ Pattern Matching for switch is still PREVIEW in Java 17 — it became stable only in Java 21.